

h e p i a



Haute école du paysage, d'ingénierie
et d'architecture de Genève

OS pour le déploiement de services conteneurisés

Défense du projet de bachelor

h e p i a

Haute école du paysage, d'ingénierie
et d'architecture de Genève

Frédéric ARROYO

2026-09-01

HEPIA // HES-SO Genève

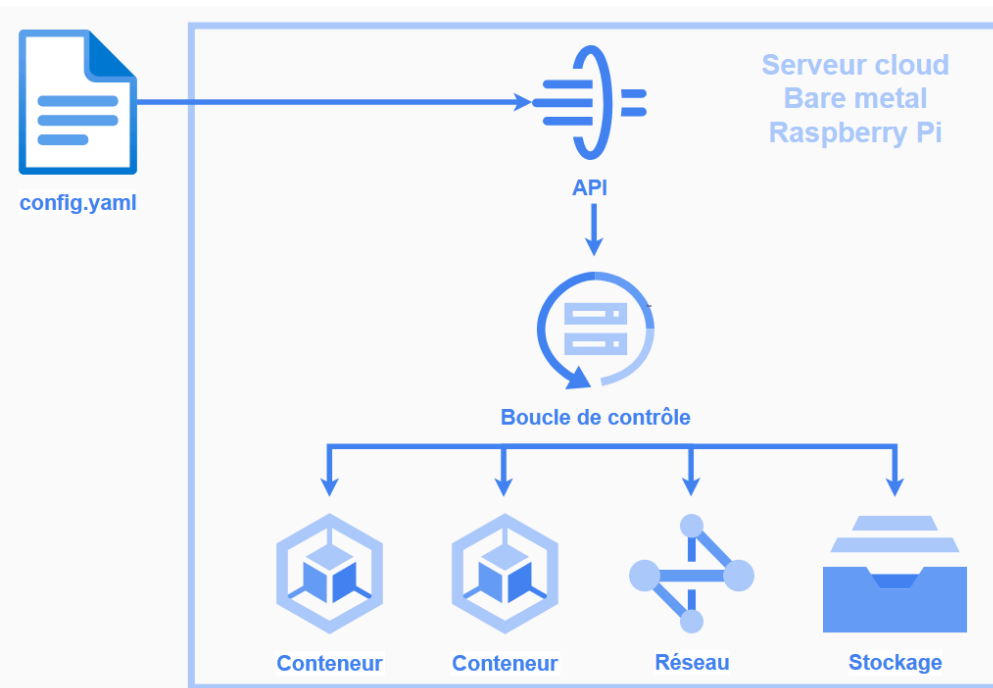
- Déploiements modestes
- Basé sur les conteneurs
- Étapes communes
 1. Installer l'OS
 2. Configurer le système (SSH, réseau, paquets)
 3. Installer le runtime de conteneur
 4. Déployer les conteneurs

Constat

Malgré ces besoins simples, aucun OS ne remplit ce rôle simplement.

- Projet de semestre, octobre $\Rightarrow$ mars)
 - Identification de solution existante adéquate: **aucune**
 - Conceptualisation de très haut niveau
 - Recherche des « briques » logicielles
- IA: tentative pour débogage + amélioration CI/CD
 - Sans succès
 - **Pas de code ou de décisions architecturales**
- Projet personnel

- Distribution Linux spécialisée
- Surface **minimale**: le strict nécessaire pour les conteneurs
 - Pas d'SSH, de shell, ou de commandes
 - Piloté par API
- Fichier de configuration unique
 - Système déclaratif
 - Homogène: bare-metal, VPS, embarqué, etc.



Démonstration

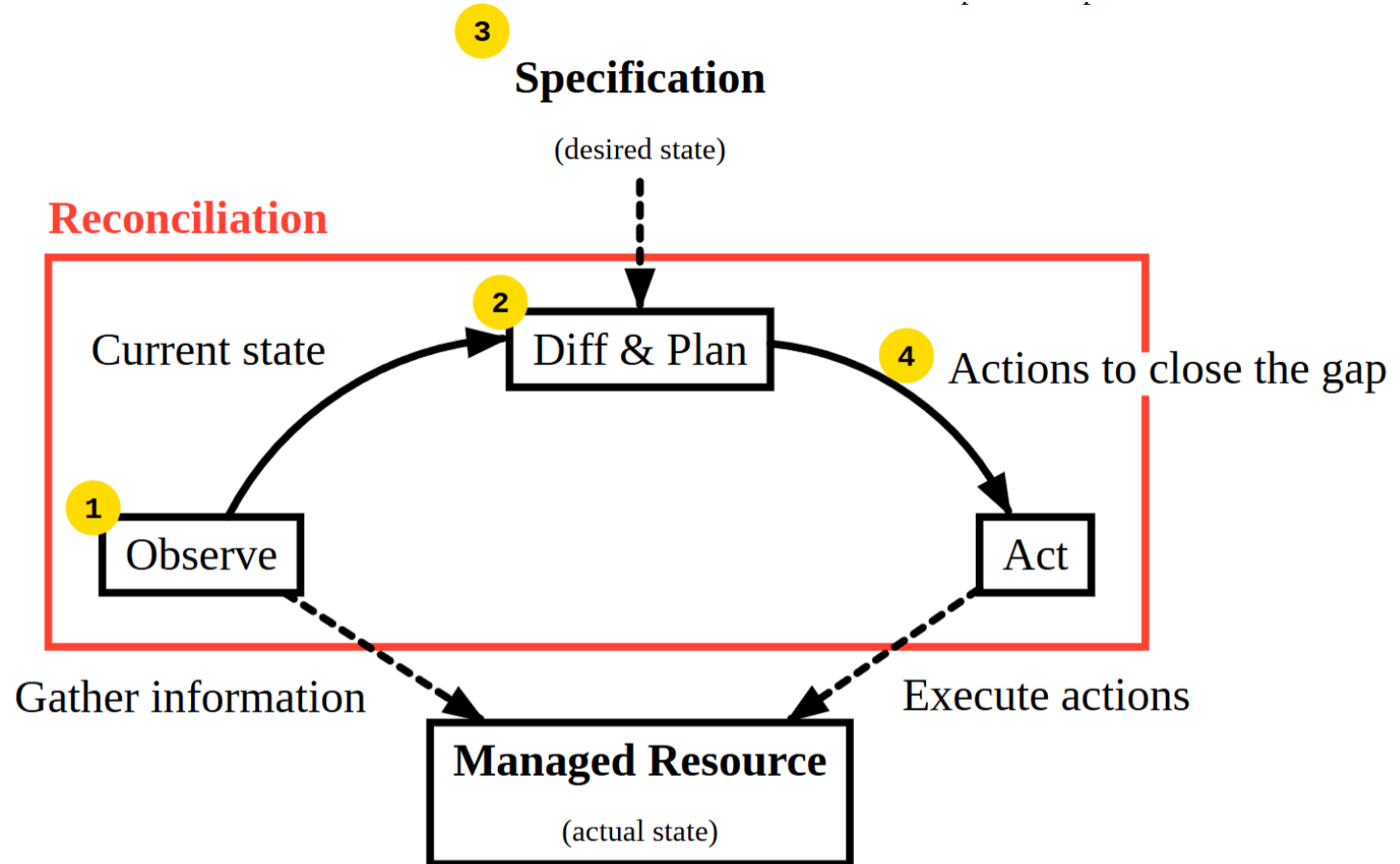


1. **Conception**
2. **Implémentation**
3. **Tests & Validation**
4. **Comparaison avec d'autres solutions**

Conception

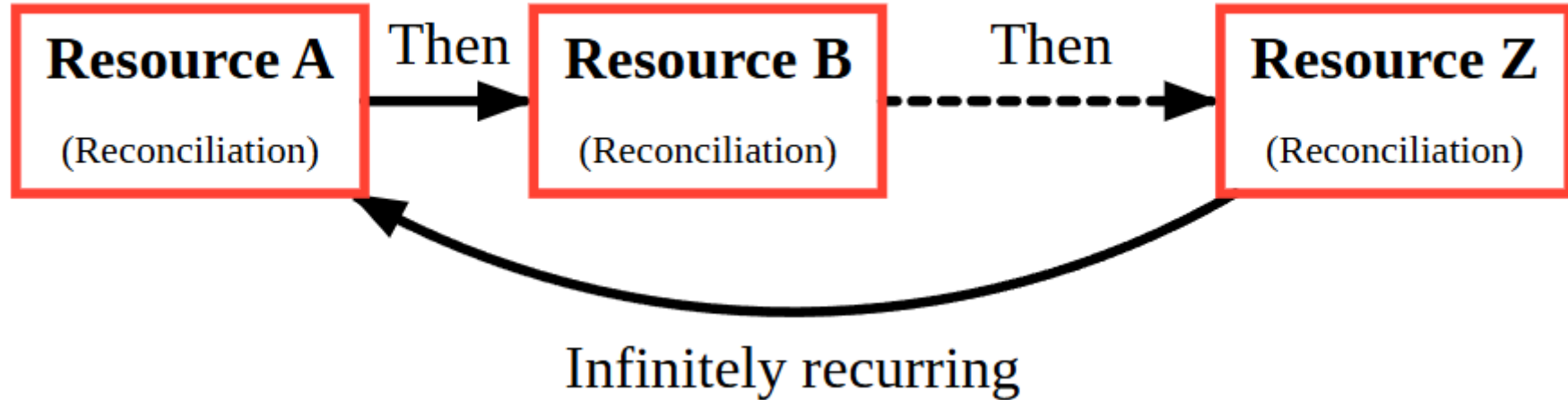


- **Définition:** une boucle qui compare l'état actuel avec l'état désiré afin de les faire correspondre



- **Ressource**: objet qui regroupe l'état désiré, et un instantané de l'état actuel
- **Contrôleur**: implémente la *réconciliation* pour une *ressource* donnée
 - Réseau
 - Conteneur
 - Système

- Décentralisé: chaque contrôleur/ressource à sa propre boucle
 - Plus flexible, mais plus compliqué à implémenter
- **Centralisé**: une boucle centrale qui contrôle tout
 - Plus rigide, mais plus simple à implémenter



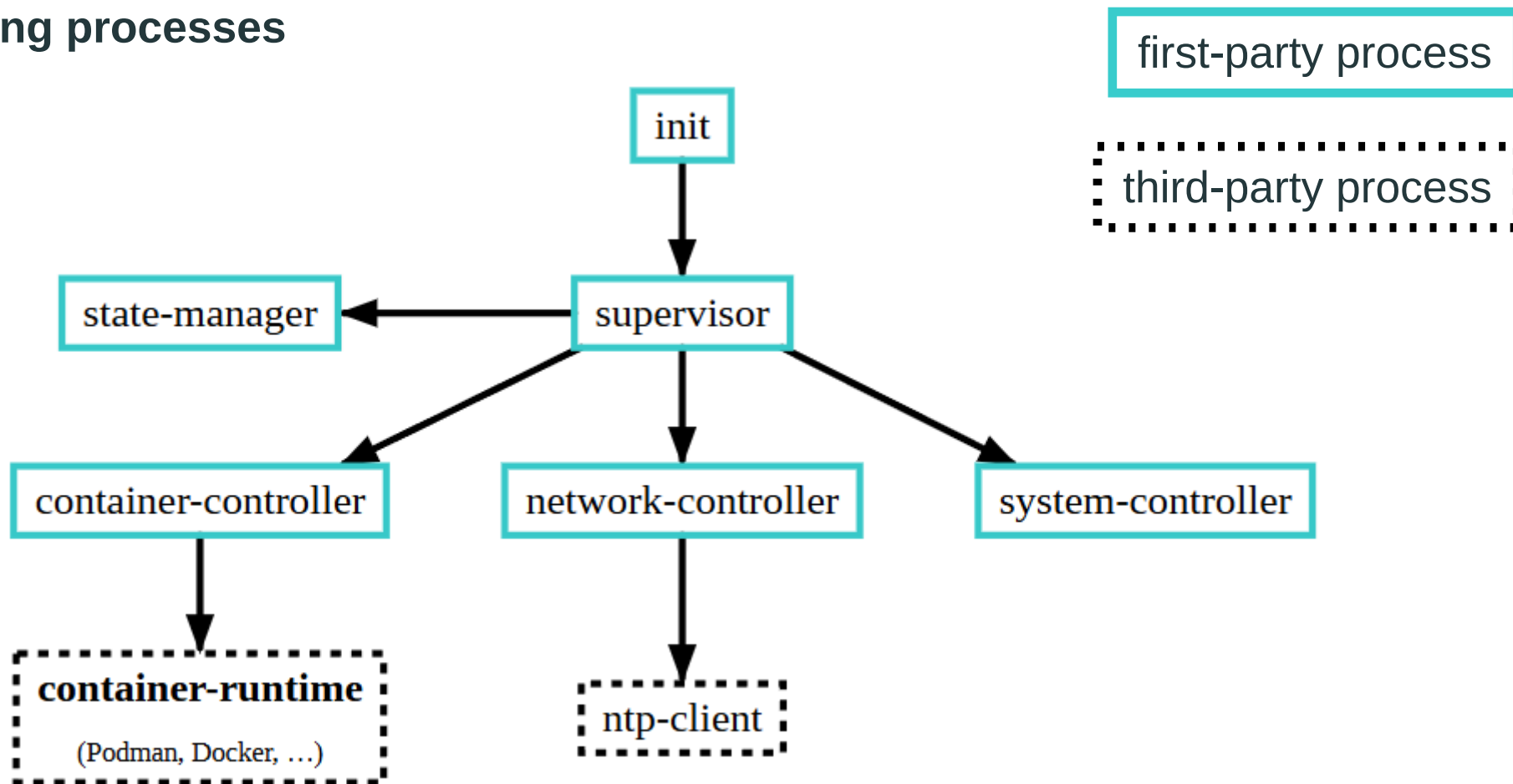
Implémentation



- Basé sur aucune distribution/programme existant
 - Tout est développé de zéro
- Langage de programmation: Rust
- Composants externes:
 - Runtime de conteneur: Podman
 - Bootloader: Limine

- Racine immuable avec couche d'écriture temporaire
 - SquashFs, Tmpfs, OverlayFs
- /config et /var persistés sur partitions ext4
- /etc et autres reconstruits à chaque redémarrage
- Persistance optionnelle: système peut être rendu entièrement éphémère

All running processes



- Compilation de multiples composants : noyau Linux, contrôleurs, init...
- Packaging en images SquashFS, assemblées en image ISO
- Cible plusieurs architectures : x64 et ARM
- Système de build: Nix
 - Builds 100% reproductibles
- Inconvénient: recompilations longues avec Rust

Test & validation



- 40 tests unitaires et intégrations
- 14 tests de bout en bout (E2E)
 - VM isolée
 - Interaction uniquement via le client d'API

- Sur 100 échantillons
- Même environnement que les tests E2E
- RAM: **160 MiB** pour un conteneur, **<80 MiB** pour le système seul
 - 160 MiB majoritairement dus à Podman
- Rapidité:
 - **2.1s** cas idéal (machine moderne)
 - 19.3s installation (machine moderne)
 - 35s cas idéal (Raspberry Pi 1B [2008])

Comparaison avec d'autres solutions

Talos Linux

- **Orienté Kubernetes**
- Déclaratif et piloté par API
- Minimaliste

NixOS

- **Générique**
- se base sur Nix
- Déclaratif, mais pas en continu
- Complexe à prendre en main

Critères	ContainerOS	NixOS	Talos
Automatisation	✓	★	✓
Mémoire requise en exécution	160 MiB	276 MiB	1.4 GiB
Mémoire requise à l'installation	160 MiB	762 MiB	1.4 GiB
Rapidité d'installation	36s	300s	210s
Rapidité de démarrage	5.6s	31s	65s
Simplicité	✓	✓	X

Conclusion



- **But:** un OS pour déployer des conteneurs de manière simple
- **Solution:** basé sur rien, piloté par API et 100% déclaratif
- **Résultats:**
 - Rapide et très léger (160 MiB)
 - Tous les objectifs ont été atteints
- **Difficultés:**
 - Bugs dans des composants externes (Podman, WSL, bibliothèque Rust)
 - Champ large
 - Évolution en terrain inconnu

- **Court terme**: mise à jour des composants + observabilité
- **Moyen terme**: plus de fonctionnalité dans les contrôleurs existants
- **Long terme**: système de plugin

- Très intéressant
 - Mise en pratique des technologies vues en cours
 - Domaine du développement jamais touché
- Projet personnel, amené à être maintenu dans le futur

Questions

h e p i a



Haute école du paysage, d'ingénierie
et d'architecture de Genève